9. Keylime TPM 유틸리티 및 암호화 기능 분석

9.1 개요

Keylime의 TPM 유틸리티와 암호화 기능은 시스템의 핵심 보안 메커니즘을 구현합니다. 이 문서는 `keylime/tpm/tpm_util.py` 와 `keylime/crypto.py` 의 주요 기능들을 분석하여 TPM 2.0 기반 검증 시스템의 암호화 구현을 상세히 설명합니다.

9.2 TPM 유틸리티 모듈 (`keylime/tpm/tpm_util.py`)

9.2.1 Microsoft TPM 2.0 참조 구현 기반

`tpm_util.py` 는 Microsoft의 TPM 2.0 참조 구현을 기반으로 하며, BSD 라이선스 하에 제공됩니다. 이 모듈은 TPM 2.0 표준을 준수하는 다양한 암호화 및 검증 기능을 제공합니다.

주요 임포트 및 의존성

```
from cryptography.hazmat import backends

from cryptography.hazmat.primitives import hashes, hmac, serialization

from cryptography.hazmat.primitives.asymmetric import ec, padding

from cryptography.hazmat.primitives.ciphers import Cipher, modes

from cryptography.hazmat.primitives.kdf.concatkdf import ConcatKDFHash

from cryptography.hazmat.primitives.kdf.kbkdf import KBKDFHMAC, CounterLocation,
```

9.2.2 핵심 함수 분석

9.2.2.1 `makecredential()` 함수 - 자격 증명 생성

`makecredential()` 함수는 TPM_MakeCredential 명령의 구현으로, EK(Endorsement Key)를 사용하여 AIK(Attestation Identity Key)에 대한 자격 증명을 생성합니다.

```
def makecredential(ek_tpm: bytes, challenge: bytes, aik_name: bytes) -> bytes:
"""TPM_MakeCredential implementation
Parameters
----------
ek_tpm: marshalled TPMT_PUBKEY
challenge: random 'password'
aik_name: name of the object (AIK)
"""
```

동작 과정:

1. **공개키 추출**: `tpm2_objects.pubkey_parms_from_tpm2b_public()` 를 통해 EK에서 공개키와 해시 알고리즘 추출
2. **암호화 방식 결정**: RSA 또는 ECC 키 타입에 따른 암호화 방식 선택
3. **시드 생성 및 암호화**:

- RSA의 경우: OAEP 패딩을 사용한 공개키 암호화
- ECC의 경우: `crypt_secret_encrypt_ecc()` 함수를 통한 ECDH 기반 암호화

4. **자격 증명 생성**: `secret_to_credential()` 함수를 통한 최종 자격 증명 블롭 생성

9.2.2.2 RSA 암호화 구현

```
if isinstance(public_key, RSAPublicKey):
random = os.urandom(hashfunc.digest_size)
```

```
secret = public_key.encrypt(
random,
padding.OAEP(
mgf=padding.MGF1(algorithm=hashfunc),
algorithm=hashfunc,
label=label_to_bytes("IDENTITY")
),
)
```

9.2.2.3 ECC 암호화 구현 (`crypt_secret_encrypt_ecc()`)

```
def  crypt_secret_encrypt_ecc(public_key: EllipticCurvePublicKey, hashfunc: hash
my_private_key = ec.generate_private_key(public_key.curve)
my_public_key = my_private_key.public_key()
point = tpm2_objects.tpms_ecc_point_marshal(my_public_key)
ecc_secret_x = my_private_key.exchange(ec.ECDH(), public_key)
# KDFe를 사용한 키 유도
data = crypt_kdfe(hashfunc, ecc_secret_x, "IDENTITY", party_x, party_y, digest_s
return data, point
```

9.2.3 키 유도 함수 (Key Derivation Functions)

9.2.3.1 `crypt_kdfa()` - TPM 2.0 KDFa 구현

```
def  crypt_kdfa(
hashfunc: hashes.HashAlgorithm,
key: bytes,
label: str,
context_u: bytes,
context_v: bytes,
size_in_bits: int,
) -> bytes:
"""TPM 2's KDFa"""
size_in_bytes = (size_in_bits + 7) >> 3
context = context_u + context_v


kdf = KBKDFHMAC(
algorithm=hashfunc,
mode=Mode.CounterMode,
length=size_in_bytes,
rlen=4,
llen=4,
location=CounterLocation.BeforeFixed,
label=bytes(label, "UTF-8"),
context=context,
fixed=None,
```

```
)
return kdf.derive(key)
```

9.2.3.2 `crypt_kdfe()` - TPM 2.0 KDFe 구현

```
def  crypt_kdfe(
hashfunc: hashes.HashAlgorithm,
secret_x: bytes,
label: str,
party_x: bytes,
party_y: bytes,
size_in_bits: int,
) -> bytes:
"""TPM 2's KDFe"""
size_in_bytes = (size_in_bits + 7) >> 3
label_bytes = label_to_bytes(label)
otherinfo = label_bytes + party_x + party_y

kdf = ConcatKDFHash(
algorithm=hashfunc,
length=size_in_bytes,
otherinfo=otherinfo,
)
```

```
return kdf.derive(secret_x)
```

9.2.4 보안 래핑 및 무결성 보호

9.2.4.1 `produce_outer_wrap()` - 암호화 후 MAC 구현

```
def  produce_outer_wrap(
ek_tpm: bytes,
name: bytes,
hashfunc: hashes.HashAlgorithm,
seed: bytes,
sensitive_data_2b: bytes,
) -> Tuple[bytes, bytes]:
"""TPM 2's ProduceOuterWrap implementing encrypt-then-MAC of a secret"""
# 1. 보호 키 계산
symkey, sym_alg = compute_protection_key_parms(ek_tpm, hashfunc, name, seed)
# 2. 대칭키 암호화 (CFB 모드)
symcipherfunc = tpm2_objects.SYMCIPHER_FUNCS.get(sym_alg)
symcipher = symcipherfunc(symkey)
block_size = symcipher.block_size >> 3
encryptor = Cipher(symcipher, modes.CFB(b"\x00" * block_size)).encryptor()
ciphertext = encryptor.update(sensitive_data_2b) + encryptor.finalize()
# 3. HMAC 기반 무결성 보호
hmac_signature = compute_outer_integrity(name, hashfunc, seed, ciphertext)
```

```
return hmac_signature, ciphertext
```

9.2.4.2 `compute_outer_integrity()` - HMAC 무결성 계산

```
def  compute_outer_integrity(
name: bytes,
hashfunc: hashes.HashAlgorithm,
seed: bytes,
ciphertext: bytes,
) -> bytes:
"""TPM 2's ComputeOuterIntegrity HMAC'ing a ciphertext"""
digest_size = hashfunc.digest_size
hmac_key = crypt_kdfa(hashfunc, seed, "INTEGRITY", b"", b"", digest_size << 3)
h = hmac.HMAC(hmac_key, hashfunc)
h.update(ciphertext)
h.update(name)
return h.finalize()
```

9.2.5 Quote 검증 기능

9.2.5.1 `checkquote()` 함수

```
def  checkquote(
aikblob: bytes, nonce: str, sigblob: bytes, quoteblob: bytes, pcrblob: bytes, ex
) -> Dict[int, str]:
```

```
"""Check the given quote by checking the signature, then the nonce and then the
```

검증 단계:

1. **서명 알고리즘 확인**: RSA/ECC 키 타입과 서명 알고리즘 호환성 검증
2. **디지털 서명 검증**: `verify()` 함수를 통한 서명 유효성 확인
3. **Nonce 검증**: 예상 nonce와 실제 nonce 비교
4. **PCR 다이제스트 검증**: PCR 값들의 해시와 quote의 다이제스트 비교

9.2.6 유틸리티 함수들

9.2.6.1 `random_password()` - 안전한 패스워드 생성

```
def random_password(length: int = 20) -> str:
rand = crypto.generate_random_key(length)
chars = string.ascii_uppercase + string.digits + string.ascii_lowercase
password = ""
for i in range(length):
password += chars[(rand[i]) % len(chars)]
return password
```

9.2.6.2 `verify()` - 통합 서명 검증

```
def verify(
pubkey: SupportedKeyTypes,
sig: bytes,
```

```
digest: bytes,
hashfunc: hashes.HashAlgorithm,
sigalg: int = tpm2_objects.TPM_ALG_RSASSA,
saltlen: int = 0,
) -> None:
"""Do signature verification with the given public key"""
```

9.3 암호화 모듈 (keylime/crypto.py)

9.3.1 RSA 공개키 암호화

9.3.1.1 키 관리 함수들

```
def rsa_import_pubkey(pubkey: Union[str, bytes]) -> RSAPublicKey:
"""Import a public key"""
def rsa_import_privkey(privkey: Union[str, bytes], password: Optional[bytes] =
"""Import a private key"""
def rsa_generate(size: int) -> RSAPrivateKey:
"""Generate private key"""
```

9.3.1.2 RSA 암호화/복호화

```
def rsa_encrypt(key: RSAPublicKey, message: bytes) -> bytes:
"""RSA encrypt message"""
```

```
return key.encrypt(
bytes(message),
OAEP(
mgf=MGF1(algorithm=hashes.SHA1()),
algorithm=hashes.SHA1(),
label=None,
),
)


def  rsa_decrypt(key: RSAPrivateKey, ciphertext: bytes) -> bytes:
"""RSA decrypt message"""
return key.decrypt(
ciphertext,
OAEP(
mgf=MGF1(algorithm=hashes.SHA1()),
algorithm=hashes.SHA1(),
label=None,
),
)
```

9.3.2 AES 대칭키 암호화

9.3.2.1 AES-GCM 암호화 구현

```
def  encrypt(plaintext: Optional[bytes], key: bytes) -> bytes:
"""Encrypt object"""
if plaintext is None:
plaintext = b""
iv = generate_random_key(AES_BLOCK_SIZE)
encryptor = Cipher(algorithms.AES(key), modes.GCM(iv), backend=default_backend()
cipher_text = encryptor.update(plaintext) + encryptor.finalize()
assert  isinstance(encryptor, AEADEncryptionContext)
return base64.b64encode(iv + cipher_text + encryptor.tag)
```

9.3.2.2 AES-GCM 복호화 구현

```
def  decrypt(ciphertext: bytes, key: bytes) -> bytes:
"""Decrypt object"""
ciphertext = base64.b64decode(ciphertext)
iv = ciphertext[:AES_BLOCK_SIZE]
tag = ciphertext[-AES_BLOCK_SIZE:]
ciphertext = bytes(ciphertext[AES_BLOCK_SIZE:-AES_BLOCK_SIZE])

decryptor = Cipher(algorithms.AES(key), modes.GCM(iv, tag), backend=default_back
return decryptor.update(ciphertext) + decryptor.finalize()
```

9.3.3 HMAC 기반 메시지 인증

9.3.3.1 HMAC 생성

```
def do_hmac(key: bytes, value: str) -> str:
"""Generate HMAC"""
h = hmac.new(key, msg=None, digestmod=hashlib.sha384)
h.update(value.encode("utf-8"))
return h.hexdigest()
```

9.3.3.2 키 유도 함수 (PBKDF2)

```
def kdf(password: str, salt: str) -> bytes:
mykdf = PBKDF2HMAC(
algorithm=hashes.SHA256(),
length=32,
salt=bytes(salt, encoding="utf8"),
iterations=100000,
backend=default_backend(),
)
return mykdf.derive(password.encode("utf-8"))
```

9.3.4 디지털 서명

9.3.4.1 RSA 서명 생성

```
def  rsa_sign(key: RSAPrivateKey, message: bytes, paddingt: str = "internal") ->

"""RSA sign message"""

_padding: Union[padding.PSS, padding.PKCS1v15]

if paddingt == "internal":

_padding = padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PS

elif paddingt == "default":

_padding = padding.PKCS1v15()

else:

raise ValueError



signature = key.sign(message, _padding, hashes.SHA256())

return base64.b64encode(signature)
```

9.3.4.2 RSA 서명 검증

```
def  rsa_verify(public_key: RSAPublicKey, message: bytes, signature: bytes) -> b

"""RSA verify message"""

try:

public_key.verify(

base64.b64decode(signature),

message,

padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGT

hashes.SHA256(),
```

```
)
except exceptions.InvalidSignature:
return False
except Exception as e:
raise e
return True
```

9.4 보안 특징 및 구현 세부사항

9.4.1 암호화 알고리즘 지원

9.4.1.1 해시 알고리즘

- **SHA-1**: 레거시 지원 (보안상 권장하지 않음)
- **SHA-256**: 기본 해시 알고리즘
- **SHA-384**: HMAC에서 사용
- **SHA-512**: 고보안 요구사항

9.4.1.2 대칭키 암호화

- **AES-128/256**: 블록 암호화
- **GCM 모드**: 인증된 암호화 (AEAD)
- **CFB 모드**: TPM 내부 암호화

9.4.1.3 공개키 암호화

- **RSA**: 2048/4096 비트 키 지원
- **ECC**: NIST P-256, P-384, P-521 곡선 지원

- **OAEP 패딩**: RSA 암호화 시 사용
- **PSS 패딩**: RSA 서명 시 사용

9.4.2 보안 고려사항

9.4.2.1 난수 생성

```
def generate_random_key(size: int = 32) -> bytes:
"""Generate random key using urandom wrapper"""
return os.urandom(size)


def get_random_bytes(size: int) -> bytes:
"""Generate random bytes"""
return secrets.token_bytes(size)
```

9.4.2.2 키 유도 및 관리

- **PBKDF2**: 패스워드 기반 키 유도 (100,000 반복)
- **KDFa/KDFe**: TPM 2.0 표준 키 유도 함수
- **HMAC-기반 KDF**: 카운터 모드 키 유도

9.4.2.3 메모리 보안

- 민감한 데이터의 안전한 처리
- 키 소재의 적절한 소거
- 타이밍 공격 방지

9.5 통합 및 상호운용성

9.5.1 TPM 2.0 표준 준수

- Microsoft TPM 2.0 참조 구현과의 호환성
- TPM 2.0 명령어 세트 지원
- 표준 데이터 구조 및 알고리즘 사용

9.5.2 Cryptography 라이브러리 활용

- Python cryptography 패키지 기반
- 하드웨어 가속 지원
- FIPS 140-2 인증 알고리즘 사용

9.5.3 에러 처리 및 예외 관리

- 암호화 연산 실패 시 적절한 예외 발생
- 입력 데이터 검증 및 타입 체크
- 보안 관련 오류의 안전한 처리

이 분석을 통해 Keylime의 TPM 유틸리티와 암호화 기능이 어떻게 구현되어 있으며, TPM 2.0 표준을 준수하면서도 강력한 보안 기능을 제공하는지 이해할 수 있습니다.